



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

MedFactCheck

Anno Accademico 2025/2026

Candidati

Gabriel Covone M63001809

Antonella Scellini M63001853

Francesco Esposito M63001830

Valerio Cera M63001700

Contents

1	Orchestratore	1
1.1	Architettura di Rete e Disaccoppiamento Client-Server	2
1.2	Il Pattern Multi-Agente (Hub & Spoke)	3
1.2.1	Gestione dello Stato Globale (MultiAgentState)	3
1.2.2	Dependency Injection e Ottimizzazione VRAM	4
1.2.3	Il Ruolo dell'Orchestratore	4
1.2.4	Il Ruolo del Supervisore	5
1.2.5	I Nodi Operativi (Spoke)	6
1.3	Flusso Tipico di Esecuzione (Workflow)	7
2	Claim Processing	10
2.1	Manuale d'Architettura: Claim Ingestion & Decomposition	10
2.2	L'Agente IngestionAndDecomposer	10
2.3	Routing Predittivo (KB vs LIT)	12
3	Retrieval	14
3.1	Architettura di Sistema e Master Orchestrator	15
3.1.1	Unificazione dei Formati di Output	16
3.2	Il Ramo KB (Knowledge Base Locale)	16

3.2.1	Zero-Latency Embedding e Allineamento NumPy 2.x	17
3.2.2	Fonte dei Dati: DisGeNET (Ingestion Locale) e Textualizzazione	17
3.2.3	Risoluzione del Type Mismatch FP16/FP32 in FAISS	18
3.2.4	Routing Condizionale e OOM-Proofing	19
3.2.5	Gestione Sicura della Cache e Design Fail-Fast	20
3.3	Il Ramo LIT (Letteratura Medica Mondiale)	20
3.3.1	Design Pattern Cache-Aside (MongoDB)	21
3.3.2	Chunking Granulare ed Esplosione del Pool	22
3.3.3	Filtrazione a Due Stadi (Funnel Pipeline)	22
3.4	Il Retriever Agent (<code>RetrievalFunc.py</code>)	23
4	Reasoning	25
4.1	Manuale d'Architettura: Reasoning Agent	25
4.1.1	L'Obiettivo: Interpretability & Chain-of-Thought (CoT)	25
4.1.2	Funzionamento dell'Agente	26
4.1.3	Sinergia con la Veracity	28
5	Veracity	29
5.1	Manuale d'Architettura: Veracity Assessment	29
5.2	Il Modello di Classificazione (Cross-Encoder NLI)	29
5.3	Le Classi di Output e la Confidenza	30
5.4	Aggregazione del Verdetto Finale (Zero-Tolerance Policy)	32

6	Storage	34
6.1	Manuale d'Architettura: Storage e Data Provenance (MedFactCheck)	34
6.2	Architettura del Database: MongoDB	34
6.2.1	final_results (Storico dei Claim e Verdeti)	35
6.2.2	evidence (Data Provenance e Tracciabilità)	35
6.2.3	papers (Cache-Aside della Letteratura Europea)	36
6.3	Persistenza dello Stato del Grafo (Checkpointing)	37
6.4	Storage Vettoriale e Controllo d'Integrità (FAISS/BM25)	37
7	Dashboard	39
7.1	MedFactCheck - Dashboard Client	39
7.2	Obiettivi e Responsabilità	39
7.3	Struttura dell'Interfaccia (Layout e Navigazione)	41
7.4	Interazione Multimodale e Casi d'Uso	42
7.4.1	Caso 1: Input Puramente Testuale (Testo Libero)	43
7.4.2	Caso 2: Input da Fonte Esterna (URL)	44
7.4.3	Caso 3: Input Puramente Visivo	46
7.4.4	Caso 4: Input Ibrido (Testo + Immagine)	48
7.5	Gestione degli Errori (Defensive Programming)	51
7.6	I Vantaggi del Disaccoppiamento	52
7.7	Strategia di Caching (Smart Keying)	53
8	Valutazione del Sistema e Conclusioni	54
8.1	Metodologia di Benchmarking	54
8.2	L'Effetto Pendolo nel Prompt Engineering	55
8.3	Conclusioni	63

Chapter 1

Orchestratore

Il modulo **Orchestrator** rappresenta il sistema nervoso centrale di **MedFactCheck**. Il suo compito è governare l'intero ciclo di vita della verifica di un claim biomedico (notizia, studio, affermazione, immagine medica, ecc.), coordinando l'esecuzione asincrona dei vari modelli di Intelligenza Artificiale, gestendo l'I/O verso il database (MongoDB) e interfacciandosi con il client esterno.

Per garantire la massima flessibilità e scalabilità, l'orchestrazione non si basa su un automa a stati finiti rigido, bensì su un'architettura **Multi-Agente dinamica (Hub & Spoke)** basata sul framework **LangGraph**, esposta nativamente tramite un'interfaccia RESTful (**FastAPI**).

1.1 Architettura di Rete e Disaccoppiamento Client-Server

Per simulare in modo accurato un ambiente di produzione *Enterprise* (e permettere il testing ottimale su infrastrutture cloud), il sistema è stato ingegnerizzato separando nettamente il livello di presentazione dal motore di calcolo:

1. **Server Back-End (`api.py` / **Orchestrator**):** Un server asincrono ad alte prestazioni basato su **FastAPI** e `uvicorn`. Ha il monopolio esclusivo sull'hardware. Rimane in ascolto sul path `/verify` accettando richieste HTTP POST asincrone contenenti payload testuali e/o buffer di immagini mediche, salvandole temporaneamente in totale sicurezza.
2. **Client Front-End (`Dashboard.py`):** Un'interfaccia ultraleggera in Streamlit che delega il carico computazionale all'API tramite chiamate `requests.post`, interrogando poi localmente MongoDB per renderizzare le metriche e i risultati in tempo reale.

Questo disaccoppiamento risponde direttamente ai requisiti di **Scalabilità**: il server può scalare verticalmente o orizzontalmente indipendentemente dal numero di dashboard connesse, elaborando *request* concorrenti senza bloccare l'interfaccia utente.

1.2 Il Pattern Multi-Agente (Hub & Spoke)

La logica decisionale risiede nel file `multi_agent.py` (e nei componenti definiti nel grafo LangGraph), che istanzia un grafo ciclico (`StateGraph`). Al centro di questo grafo siede l'Agente **Supervisor** (l'Hub), circondato dai nodi operativi (gli Spoke: `Decomposer`, `Retriever`, `Reasoner`, `Veracity`).

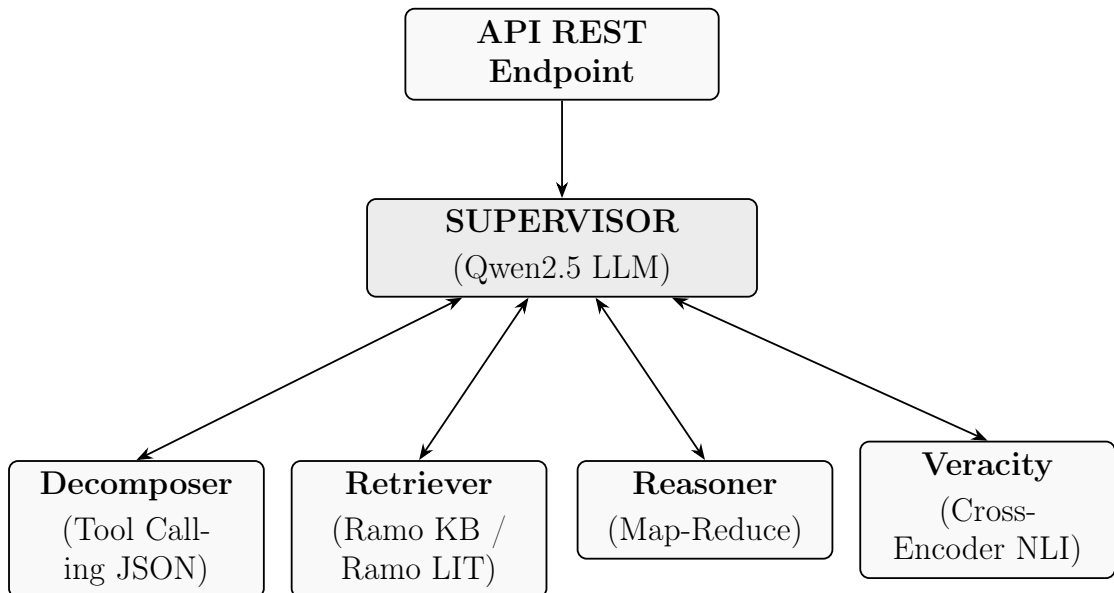


Figure 1.1: Pattern Multi-Agente (Hub & Spoke): Il Supervisore orchestra in modo disaccoppiato i 4 nodi operativi indipendenti, aggiornando iterativamente lo stato globale.

1.2.1 Gestione dello Stato Globale (MultiAgentState)

Nei sistemi multi-agente, la comunicazione non avviene tramite chiamate dirette tra funzioni, ma attraverso la mutazione di uno stato condiviso. Il sistema definisce un `TypedDict` rigoroso che fa da

"memoria a breve termine". Questo stato contiene la storia conversazionale (`messages`), l'input multimodale (`claim_input`), e chiavi specifiche che i singoli nodi popolano in sequenza (`sub_claims`, `retrieved_docs`, `reasoning_output`, `veracity_results`).

1.2.2 Dependency Injection e Ottimizzazione VRAM

Per prevenire la frammentazione della VRAM e il rischio di Out of Memory (OOM), i modelli Deep Learning (Qwen in formato NF4, DeBERTa, e il SentenceTransformer in FP16) vengono istanziati una sola volta a livello globale all'avvio del server. I puntatori a queste istanze vengono poi iniettati nelle classi degli agenti, garantendo che le risorse hardware siano condivise in modo efficiente tra tutti i thread operativi.

1.2.3 Il Ruolo dell'Orchestratore

L'**Orchestratore** è il macro-componente che incapsula le API FastAPI, le connessioni ai database e l'istanza del grafo LangGraph. Si occupa di:

- Ricevere le richieste di fact-checking dal client e validare gli input (es. testo del claim o eventuali allegati).
- Gestire la persistenza dello stato e la *Fault Tolerance* (tramite `langgraph-checkpoint-mongodb`).

- Fornire e allocare le risorse hardware necessarie agli agenti (modelli AI in esecuzione locale o remota).
- Raccogliere e confezionare il responso finale formattato da restituire al client.

Nota sull'Esecuzione: Mentre l'endpoint FastAPI è puramente asincrono (`async def`), l'esecuzione dei nodi di LangGraph e gli accessi CRUD su MongoDB (PyMongo) avvengono in modo deterministico e sincrono. Questa scelta architetturale previene *race-condition* durante l'aggiornamento simultaneo dei sub-claims sul database.

1.2.4 Il Ruolo del Supervisore

Il **Supervisor** è un agente primario, mosso da un LLM avanzato (Qwen2.5), specializzato nel ragionamento orchestrato e nel routing dinamico. Si occupa di:

1. **Analisi Iniziale e Delega:** Non appena il claim entra nel grafo, viene valutato dal Supervisor. Questo decide quale "specialista" (Spoke) chiamare in base al task da compiere.
2. **Controllo dello Stato:** Al termine del lavoro di ciascun nodo operativo, il flusso ritorna obbligatoriamente al Supervisor. Esso valuta l'output ricevuto dallo specialista e decide la prossima mossa.

3. **Terminazione:** Una volta che l'intero workflow porta a un risultato solido (giudizio calcolato sul claim), il Supervisor dichiara chiusa l'esecuzione, facendo in modo che lo stato finale esca dal grafo.

1.2.5 I Nodi Operativi (Spoke)

- **Ingestion & Decomposer:** Modulo multimodale (Vision-Language). Si attiva per analizzare testi complessi, articoli via URL o referti medici per immagini. Li suddivide in *sub-claims* atomici e testabili singolarmente (JSON) invocando dinamicamente strumenti esterni (Tool Calling).
- **Retriever:** L'agente di ricerca delle evidenze scientifiche/mediche a supporto o smentita del claim. In MedFactCheck può operare lungo due direttrici:
 - **Ramo KB:** Interroga Knowledge Base interne altamente strutturate sfruttando le potenzialità della Retrieval-Augmented Generation (RAG) mediante indici vettoriali (es. FAISS) o classici (BM25).
 - **Ramo LIT:** Raccoglie evidenze dalla letteratura e database esterni interrogabili se i documenti locali non sono sufficienti.
- **Reasoner:** Riceve in input le prove fornite dal Retriever as-

sociate al claim. Attraverso pattern come *Map-Reduce*, ragiona sui dati testuali, filtrando le informazioni rumorose, per generare una sintesi delle evidenze mirata e unificata.

- **Veracity:** Modulo finale di classificazione e allineamento (basato su un modello Cross-Encoder dedicato DeBERTa). Prende in esame l'accoppiamento semantico tra [Claim] e [Sintesi delle Evidenze] restituendo una *label* di veridicità (*Supported*, *Refuted*, *Not Enough Info*) e il relativo punteggio di confidenza.

1.3 Flusso Tipico di Esecuzione (Workflow)

Per comprendere meglio come i componenti collaborino in sincrono, di seguito il ciclo di vita standard di una richiesta:

1. **Richiesta Iniziale:** L'utente inserisce un claim testuale o carica un'immagine sulla dashboard, che invia una POST all'Orchestratore (API).
2. **Presa in Carico e Stato Iniziale:** L'Orchestratore avvia la macchina LangGraph. Gli input grezzi (testo o path immagini) vengono inseriti nel `MultiAgentState` iniziale.
3. **Valutazione Iniziale (Supervisor):** Il Supervisor ispeziona lo stato (inizialmente privo di operazioni pregresse) e deduce che

l'input grezzo deve essere processato. Smista il controllo al nodo Decomposer.

4. **Ingestion & Decomposizione (Decomposer)**: Il nodo sceglie autonomamente quali tool eseguire (es. web scraping, validazione visiva). Successivamente scompone l'input in *sub-claims* atomici, assegnando a ciascuno una direttiva di ricerca (*routes*). Il flusso torna al Supervisor.
5. **Recupero Evidenze (Supervisor $\rightarrow$ Retriever)**: Il Supervisor legge i *sub-claims* estratti e demanda al Retriever il recupero dati. Il Retriever lancia thread paralleli per interrogare le Knowledge Base e la letteratura medica, salvando i documenti pertinenti per ogni sub-claim. Il flusso torna al Supervisor.
6. **Ragionamento (Supervisor $\rightarrow$ Reasoner)**: Il Supervisor verifica che le evidenze siano state recuperate e incarica il Reasoner di analizzarle. Quest'ultimo applica il pattern Map-Reduce per condensare il significato profondo dei documenti recuperati, producendo una *Chain-of-Thought* (CoT). Il flusso torna al Supervisor.
7. **Giudizio NLI (Supervisor $\rightarrow$ Veracity)**: Il Supervisor invia i risultati elaborati al nodo finale. Veracity usa il Cross-Encoder per allineare semanticamente i claim alle prove e ai ragionamenti, emettendo la sentenza finale (es. "Refuted" al 99.2%). Il flusso

torna al Supervisor.

8. **Terminazione e Salvataggio (Supervisor $\rightarrow$ FINISH):** Il Supervisor riconosce che tutti i passaggi logici sono stati completati con successo e chiude il grafo. Durante l'intero ciclo, l'Orchestratore salva progressivamente i log e i verdeti su MongoDB, restituendo infine l'ID definitivo alla Dashboard.

Chapter 2

Claim Processing

2.1 Manuale d'Architettura: Claim Ingestion & Decomposition

Il modulo di **Claim Ingestion & Decomposition** accoglie l'input dell'utente (di qualsiasi natura) e lo trasforma in una serie di *sub-claims* atomici e interrogabili.

2.2 L'Agente IngestionAndDecomposer

L'agente è mosso dal LLM **Qwen2.5-VL-7B-Instruct**, configurato per operare nativamente in modalità Multimodale tramite quantizzazione NF4 a 4-bit (ottimizzando l'occupazione in VRAM).

Fasi Operative

Fase 1: Tool Selection (Routing Dinamico dell'Input)

L'agente ispeziona l'input grezzo (un testo, un link web, o il percorso di un'immagine temporanea). Sfruttando le sue capacità decisionali e il pattern di tool-calling, restituisce un **Array JSON** contenente gli strumenti necessari, permettendo esecuzioni parallele (es. scaricare un articolo e contemporaneamente analizzare un referto allegato). I tool disponibili sono:

- `scrape_text_from_url`: Se rileva un link, scarica e pulisce il contenuto HTML della pagina web.
- `validate_image`: Se rileva il path di un'immagine, la valida per l'analisi visiva.
- `process_plain_text`: Se l'utente ha inserito testo libero o un normale claim medico, lo identifica esplicitamente per l'analisi testuale.

Fase 2: Elaborazione Multimodale e Anti-Bias (Vision-Language)

Se l'input include un'immagine medica (es. radiografie, grafici di paper, screenshot di social media), il modello utilizza la sua architettura *Vision* integrata per "guardare" l'immagine ed estrarne il contesto senza bisogno di moduli OCR esterni.

Risoluzione delle Contraddizioni (Anti-Bias): Tramite *Few-Shot Prompting* dedicato, l'agente è stato specificamente addestrato a riconoscere le discrepanze. Se l'utente fornisce un testo fuorviante (es. "Questa lesione è innocua") ma l'immagine mostra chiaramente una patologia (es. un Melanoma), il modello dà priorità all'evidenza visiva, estraendo la contraddizione come sub-claim indipendente per permetterne la smentita.

Fase 3: Decomposizione (Atomic Claim Extraction)

I testi complessi o ambigui vengono scomposti in **proposizioni dichiarative indipendenti** (Soggetto + Verbo + Oggetto). Per prevenire il troncamento della generazione e i crash per *Token Limit* (frequentissimi con input troppo lunghi come intere pagine Wikipedia), l'agente è limitato all'estrazione di un **massimo di 5 sub-claims principali**, filtrando automaticamente i dettagli irrilevanti.

2.3 Routing Predittivo (KB vs LIT)

Durante la scomposizione, l'LLM analizza la natura semantica di ogni singolo sub-claim e applica regole di classificazione severe per decidere a quale modulo di ricerca affidarlo:

- **Rotta ["kb"]:** Assegnata a definizioni biologiche, tassonomiche o di composizione statica (es. "L'aspirina è un antinfiammatorio").

rio”). Il Retriever interrogherà solo la Knowledge Base (DisGeNET).

- **Rotta ["kb", "lit"]**: Assegnata ad azioni cliniche, correlazioni o cause-effetto (es. “L’aspirina riduce il rischio di infarto”). Verrà interrogata sia la KB che la letteratura mondiale (Europe PMC).

L’output finale dell’agente è un JSON strutturato e pronto per essere passato al Retriever Agent.

Chapter 3

Retrieval

Il modulo di Retrieval costituisce il nucleo di Information Retrieval (IR) ad alte prestazioni del sistema Multi-Agente **MedFactCheck**. Il suo obiettivo primario è l'estrazione di evidenze cliniche dense, accurate e prive di rumore a partire da due sorgenti distinte: la Knowledge Base locale strutturata **DisGeNET** (Ramo KB) e la letteratura medica mondiale tramite l'API di **Europe PMC** (Ramo LIT).

L'intero modulo è orchestrato secondo un design pattern a imbuto (*funnel*) e centralizzato sotto un unico punto d'accesso ottimizzato per operare in ambienti con risorse hardware vincolate.

3.1 Architettura di Sistema e Master Orchestrator

Il sistema adotta il pattern architetturale **Facade** attraverso la classe centralizzata `MedFactCheckRetriever`. Questa scelta risponde alla necessità di disaccoppiare la logica di decomposizione dei claim (gestita dall'LLM a monte) dalle minuzie implementative dei singoli nodi di ricerca.

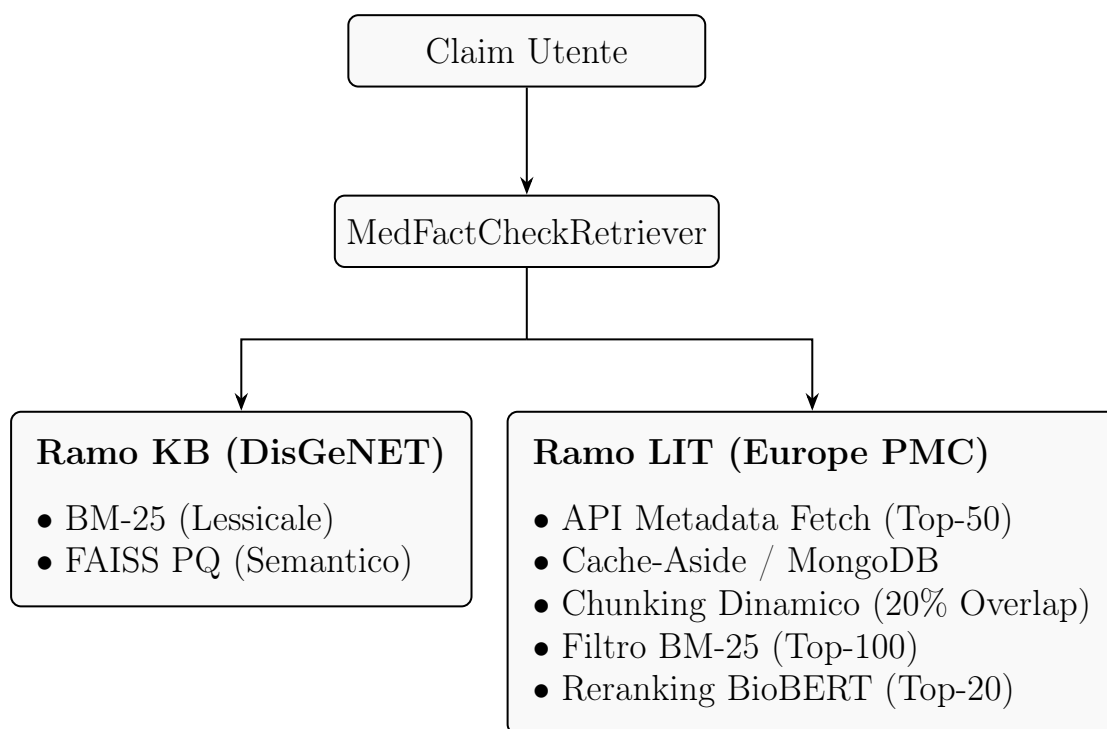


Figure 3.1: Architettura del modulo di Retrieval e routing strategico delle fonti mediche.

3.1.1 Unificazione dei Formati di Output

Nelle prime fasi di sviluppo, il Ramo KB restituiva liste di stringhe grezze, mentre il Ramo LIT produceva dizionari strutturati dotati di metadati e punteggi di confidenza. Questa eterogeneità avrebbe causato il fallimento dei parser semantici dei modelli di ragionamento (Qwen) e classificazione (DeBERTa) a valle.

Il codice è stato standardizzato affinché ogni singolo nodo di retrieval restituisca un formato JSON omogeneo composto da tre chiavi tassative:

- `text`: Lo snippet testuale normalizzato e validato.
- `source`: La tracciabilità esatta della fonte (es. KB (DisGeNET – FAISS) o PMC ID: XXX).
- `score`: Il punteggio matematico normalizzato assegnato dall’algoritmo di ranking.

3.2 Il Ramo KB (Knowledge Base Locale)

Il Ramo KB interroga la Knowledge Base **DisGeNET** combinando metodologie di ricerca sparse (lessicali) e dense (semantiche), si occupa di risolvere affermazioni dogmatiche o nozioni statiche. Quando la rotta è [KB], si utilizza il BM-25, quando invece la rotta scelta è [KB,LIT], si utilizza FAISS.

3.2.1 Zero-Latency Embedding e Allineamento NumPy

2.x

Il caricamento iterativo in memoria dei pesi dei modelli linguistici rappresenta uno dei principali colli di bottiglia di I/O nei sistemi RAG di vecchia concezione. MedFactCheck risolve questo problema istanziando il modello asimmetrico `pritamdeka/S-PubMedBert-MS-MARCO` **una sola volta all'avvio dell'applicazione** attraverso la *Dependency Injection*. Il modello viene forzato in mezza precisione (`torch.float16`), riducendo l'impronta in VRAM da ~ 440 MB a soli ~ 220 MB.

Perché è stato necessario correggere l'ambiente software:

Durante l'aggiornamento dei runtime di Google Colab a NumPy 2.x, le vecchie distribuzioni binarie di FAISS causavano crash sistematici a causa di incompatibilità dell'interfaccia C-API. Il sistema è stato stabilizzato forzando l'installazione di `faiss-gpu-cu12` unificata sotto CUDA 12, garantendo la perfetta convivenza tra NumPy 2.x e l'accelerazione hardware delle matrici di embedding.

3.2.2 Fonte dei Dati: DisGeNET (Ingestion Locale) e Textualizzazione

Per garantire assoluta stabilità in ambienti cloud (come Google Colab) e aggirare i severi firewall anti-bot (es. Cloudflare) che bloccano i download automatizzati, il Ramo KB è stato re-ingegnerizzato per

operare tramite un'**Ingestion Locale (Offline)**.

Il sistema legge direttamente dalla directory di progetto (`Disgenet/curated_gene_disease_associations.tsv`). I dati tabellari originali (relazioni validate tra geni e malattie) vengono decodificati in tempo reale e convertiti in **frasi strutturate di senso compiuto in inglese** (es. *"The gene BRCA1 is associated with the disease breast cancer with a score of 0.8..."*). Questo permette ai modelli semantici (Faiss e BM25) di "leggere" una Knowledge Base tabellare come se fosse un libro discorsivo.

3.2.3 Risoluzione del Type Mismatch FP16/FP32 in FAISS

Perché è stata inserita la conversione esplicita: Sebbene l'allineamento dell'embedder globale in `Float16` sia ideale per minimizzare l'uso della VRAM, la libreria open-source FAISS (scritta in C++ nativo) richiede tassativamente array in precisione singola standard a 32-bit (`float32`) per le sue funzioni interne di normalizzazione vettoriale e per l'algoritmo di compressione *Product Quantization* (IndexPQ). Il passaggio diretto dei vettori FP16 sollevava un errore bloccante di tipo `TypeError: argument 3 of type 'float *'`. La criticità è stata superata iniettando un casting esplicito all'interno delle funzioni di indicizzazione e di ricerca vettoriale (`search_faiss_pq`):

```
embeddings = embeddings.astype('float32')
```

```
faiss.normalize_L2(embeddings)
```

Questa soluzione coniuga i benefici di entrambi i mondi: il modello linguistico risiede stabilmente in VRAM a basso impatto (FP16), mentre il motore geometrico lavora in totale sicurezza matematica (FP32).

3.2.4 Routing Condizionale e OOM-Proofing

A seconda dei parametri logici della query, il sistema devia il flusso informativo:

- **Rotta Lessicale Esatta (**BM-25Okapi**)**: Dedicata alla verifica di nomenclature rigide o claim puramente assiomatici, operando mediante un indice invertito residente in RAM.
- **Rotta Semantica Densa (**FAISS PQ**)**: Dedicata alla comprensione di sinonimi ed espressioni discorsive, calcolando il prodotto interno (METRIC_INNER_PRODUCT) su vettori normalizzati L2 (equivalente algebrico della *Cosine Similarity*).

Per prevenire i crash da *Out Of Memory* (OOM) sui classificatori finali a complessità quadratica $O(n^2)$ (come DeBERTa), è stato inserito un algoritmo di **Hard Truncation Dinamico**. Ogni chunk estratto dalla KB viene troncato rigidamente a un massimo di 500 caratteri utilizzando una funzione `rsplit` in corrispondenza dell'ultimo spazio vuoto, preservando la coerenza sintattica dello snippet ed eliminando il testo ridondante.

3.2.5 Gestione Sicura della Cache e Design Fail-Fast

Per garantire la massima affidabilità, evitare crash da file vettoriali corrotti e mantenere l'integrità dei dati, il modulo KB implementa un rigoroso sistema di controlli:

- **Triplo Backup e Validazione Dimensionale:** Al caricamento, il sistema verifica che la dimensione degli indici BM25 (`corpus_size`) e FAISS (`ntotal`) coincida esattamente con il numero di documenti effettivamente letti da DisGeNET. In caso di corruzione, scarta la cache e passa ai file di backup secondari.
- **Fail-Fast Integrato:** Se i file sorgente locali `.tsv` risultano mancanti e non è possibile generare la conoscenza di base da zero, il sistema rifiuta di partire con dati fittizi e lancia una `RuntimeError` bloccante, preservando in modo categorico la validità della *Data Provenance*.

3.3 Il Ramo LIT (Letteratura Medica Mondiale)

Il Ramo LIT interroga la base documentale di **Europe PMC** per validare trial clinici, interazioni farmacologiche o linee guida aggiornate

in tempo reale. Dovendo elaborare articoli integrali (*Full-Text*) che superano frequentemente le 10.000 parole, il modulo adotta una rigorosa strategia a imbuto.

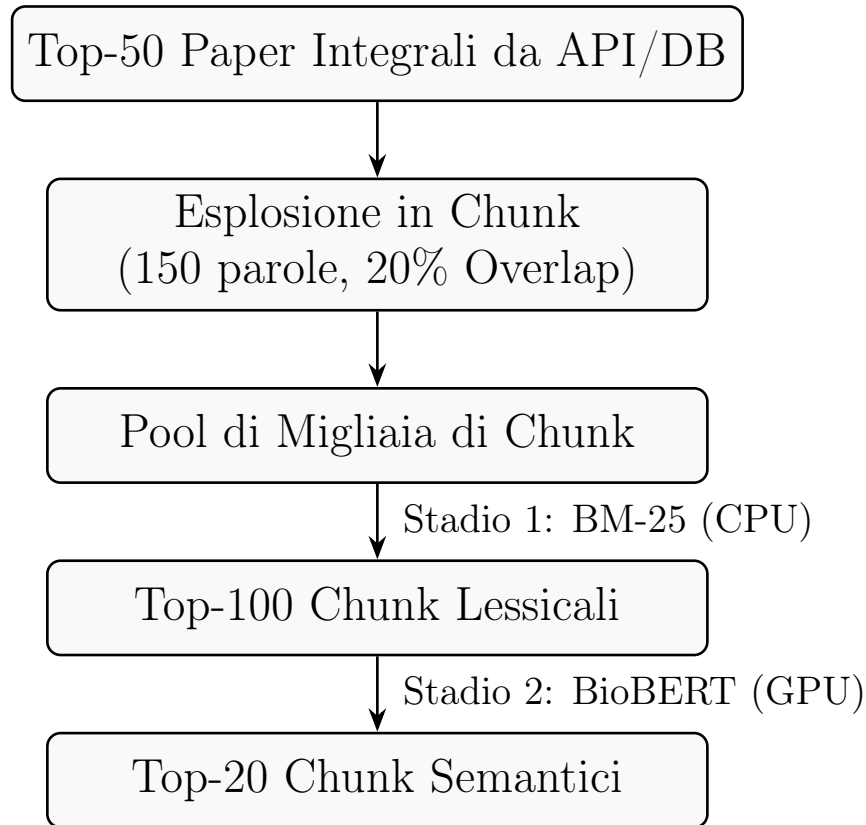


Figure 3.2: Pipeline a imbuto (Funnel Pipeline) per la filtrazione della letteratura medica.

3.3.1 Design Pattern Cache-Aside (MongoDB)

Per azzerare l'elevata latenza di rete associata al download sequenziale dei testi medici, l'architettura si appoggia a **MongoDB** secondo il pattern *Cache-Aside*. Il sistema esegue una chiamata iniziale leggerissima (`resultType: lite`) per scaricare esclusivamente i metadati e gli ID dei primi 50 articoli pertinenti.

L'istanza MongoDB locale (`medfactcheck.papers`) salva integralmente i file XML estratti. Questo garantisce tempi di risposta a latenza zero per i paper già consultati ed evita il ban delle API. Se un documento non è nel DB, viene scaricato tramite **Thread Pool concorrente** (multi-threading) e salvato asincronamente.

3.3.2 Chunking Granulare ed Esplosione del Pool

I documenti integrali (estratti da cache o scaricati in caso di Cache Miss) vengono atomizzati in blocchi di **150 parole**. Al fine di preservare il nesso causale e clinico delle frasi a cavallo tra due blocchi, viene applicato un **Overlap del 20%** (~ 30 parole). Questo processo converte i 50 articoli in un pool temporaneo di svariate migliaia di chunk candidati.

3.3.3 Filtrazione a Due Stadi (Funnel Pipeline)

Il pool massivo di migliaia di chunk viene scremato attraverso un processo bifasico sequenziale:

- **Stadio 1 (Filtro CPU Grossolano - Sparse Retrieval):**

L'intero database temporaneo viene indicizzato al volo tramite `BM25Okapi`. L'algoritmo calcola la densità lessicale esatta rispetto al claim e seleziona istantaneamente i **top-100 chunk**, scartando tutto il rumore editoriale, metodologico o bibliografico.

- **Stadio 2 (Reranking GPU Fine - Dense Retrieval):** I 100 superstiti vengono inviati in modalità *Batched Inference* all'embedder persistente in mezza precisione. Sfruttando le funzioni vettoriali native di PyTorch (`F.cosine_similarity`) sotto regime di `torch.no_grad()`, viene calcolata la vicinanza concettuale profonda, isolando i **top-20 chunk definitivi**.

Grazie a questa attenta ingegnerizzazione a imbuto, l'LLM a valle (Qwen) riceve un contesto sintetico, ad altissima densità informativa ed epidemiologica, strutturato in modo ideale per avviare le sue routine di *Inferenza Gerarchica Adattiva*.

3.4 Il Retriever Agent (**RetrievalFunc.py**)

L'orchestrazione pratica del recupero è affidata al `RetrieverAgent`. Invece di un flusso rigido in cui ogni ricerca esegue tutto, questo agente implementa un pattern **Guided Execution Multi-Tool**:

- Sfruttando le *routes* decise dal `Decomposer` (es. `["kb", "lit"]` o `["kb"]`), mappa i task sui vari Tool specifici (`kb_bm25`, `kb_faiss`, `lit_europe_pmc`).
- Lancia **ricerche parallele** per ogni singolo sub-claim (tramite `ThreadPoolExecutor`), accelerando enormemente il throughput complessivo.

- Dispone di un sistema di **Fallback Globale**: se un sub-claim non trova nulla in una determinata rotta precalcolata dal Decomposer, il sistema espande automaticamente la ricerca in extremis su tutti i database per evitare *false negative*.

Chapter 4

Reasoning

4.1 Manuale d’Architettura: Reasoning Agent

Il modulo di **Reasoning** (Ragionamento) interviene nella pipeline di MedFactCheck subito dopo il recupero delle evidenze scientifiche. Il suo ruolo è colmare il divario semantico tra i documenti grezzi recuperati e il claim originale da verificare.

4.1.1 L’Obiettivo: Interpretability & Chain-of-Thought (CoT)

In molte pipeline standard di Retrieval-Augmented Generation (RAG), le evidenze vengono passate direttamente a un classificatore o mostrate grezze all’utente. Questo approccio a “scatola nera” non fornisce all’utente alcuna spiegazione sul *perché* una certa evidenza smentisca o supporti un claim.

Il Reasoning Agent di MedFactCheck risolve questo problema generando una **Chain-of-Thought (CoT)**, ovvero un paragrafo logico e discorsivo che emula il ragionamento deduttivo di un analista biomedico esperto.

4.1.2 Funzionamento dell'Agente

L'agente (basato su un prompt specifico inviato a **Qwen2.5-VL-7B-Instruct**) riceve in input due elementi:

1. Il singolo **sub-claim** atomico prodotto dal Decomposer.
2. I **top-k paragrafi di evidenza** (chunk) recuperati e rerankati dal Retriever.

Attraverso un prompt strutturato, al modello viene richiesto di sintetizzare i risultati clinici riportati nell'evidenza, relazionarli esplicitamente con il claim e mettere in risalto discrepanze di magnitudo, iperboli, correlazioni errate o conferme empiriche. Per favorire un output più narrativo e articolato rispetto all'estrazione rigida, il parametro di *temperature* del modello viene specificamente innalzato a 0.3.

2.1 Architettura Map-Reduce (Inferenza Gerarchica)

Una delle sfide più complesse nell'uso dei LLM per il reasoning documentale è la cosiddetta *Lost in the Middle*: quando vengono forniti troppi documenti contemporaneamente, il modello tende a dimenti-

care le informazioni centrali. Per ovviare a questo problema e scalare su un numero arbitrario di evidenze recuperate, l'agente Reasoner implementa un rigoroso pattern **Map-Reduce iterativo**:

- **Chunking delle Evidenze:** I documenti recuperati vengono suddivisi in *batch* di grandezza fissa (max 4 documenti per volta).
- **Distillazione (Map):** Il modello genera un'analisi intermedia per ciascun batch, condensando le informazioni chiave e scaricando il rumore.
- **Sintesi Finale (Reduce):** Le analisi intermedie diventano il nuovo input per un'ulteriore generazione, distillando la conoscenza a piramide fino a produrre la singola Chain-of-Thought definitiva.

2.2 Short-Circuit per Assenza di Evidenze (Edge Case)

Per ottimizzare i tempi di calcolo e non gravare sull'hardware per task impossibili, l'agente implementa un controllo preventivo (*Defensive Programming*). Se il Retriever restituisce 0 documenti per un determinato sub-claim, il Reasoner salta completamente la chiamata al modello linguistico. Invece, genera in modo deterministico una stringa standard di “*Not Enough Information (NEI)*”, passando immediatamente il controllo al nodo successivo.

4.1.3 Sinergia con la Veracity

L'output discorsivo prodotto dal Reasoner viene salvato in MongoDB per la trasparenza lato Dashboard, e soprattutto viene **passato come contesto arricchito** al modulo di Veracity. Il classificatore NLI finale non deve più capire da solo testi medici complessi, ma può basarsi sull'analisi logica già pre-masticata dal Reasoner, aumentando drasticamente l'accuratezza predittiva.

Chapter 5

Veracity

5.1 Manuale d'Architettura: Veracity Assessment

Il modulo di **Veracity** rappresenta l'ultimo stadio logico della pipeline di MedFactCheck. Il suo compito è emettere una sentenza definitiva (un'etichetta di classificazione) e un punteggio di confidenza per ogni sub-claim e, infine, calcolare in maniera deterministica il verdetto globale per il claim originale dell'utente.

5.2 Il Modello di Classificazione (Cross-Encoder NLI)

Per questo compito discriminativo, il sistema abbandona l'approccio generativo (LLM) per appoggiarsi a un'architettura specializzata in

Natural Language Inference (NLI), nello specifico **DeBERTa-v3**, caricato globalmente tramite Dependency Injection per ottimizzare la VRAM.

I modelli Cross-Encoder calcolano l'allineamento semantico processando simultaneamente due sequenze di testo attraverso i layer di self-attention, restituendo logit probabilistici per tre classi fisse. Nel contesto di MedFactCheck, l'input è così strutturato:

- **Premessa (Premise):** La concatenazione delle evidenze scientifiche recuperate dal Retriever e il Ragionamento Logico (*Chain-of-Thought*) generato dal Reasoner.
- **Ipotesi (Hypothesis):** Il singolo sub-claim atomico da verificare.

Questa architettura ibrida (LLM per il reasoning + Cross-Encoder per la classificazione) garantisce che il verdetto non sia soggetto alle tipiche allucinazioni generative, ma sia il risultato di un calcolo probabilistico rigoroso.

5.3 Le Classi di Output e la Confidenza

L'agente di Veracity classifica la relazione logica in una di queste tre categorie:

- **Supported (Entailment):** Le evidenze e il ragionamento logico

confermano pienamente e inequivocabilmente le affermazioni del sub-claim.

- **Refuted (Contradiction):** Le evidenze smentiscono il sub-claim, oppure dimostrano che contiene esagerazioni critiche o palesi falsità mediche.
- **Not Enough Information / NEI (Neutral):** I documenti recuperati non trattano l'argomento in modo diretto, non sono conclusivi, o non offrono prove statistiche sufficienti per confermare o smentire il claim in modo oggettivo.

Confidence Score

Oltre all'etichetta categorica, il modello restituisce una probabilità matematica generata dall'applicazione della funzione Softmax sui logit finali (es. 0.98, ovvero 98%). Questo valore indica quanto il modello è "sicuro" della classificazione assegnata. La Dashboard sfrutta questo dato per filtrare o evidenziare risultati incerti, permettendo al clinico di valutare l'affidabilità della predizione.

5.4 Aggregazione del Verdetto Finale (Zero-Tolerance Policy)

Poiché un claim utente complesso (es. *“L’aspirina cura il cancro e l’ibuprofene causa infarti”*) viene frammentato in molteplici sub-claims dal Decomposer, l’agente di Veracity deve sintetizzare un **verdetto globale** per la dashboard.

La logica di aggregazione implementata non è una semplice media, ma segue regole cliniche conservative a “tolleranza zero”:

1. **Priorità alla Smentita:** Se *almeno un* sub-claim critico risulta **Refuted**, l’intero claim globale viene immediatamente etichettato come **Refuted**. Il razionale medico è che una notizia contenente anche solo una parziale falsità pericolosa deve essere scartata in toto.
2. **Unanimità per il Supporto:** Se tutti i sub-claims risultano **Supported**, il claim globale è approvato come **Supported**.
3. **Neutralità di Fallback:** In caso di mancanza di evidenze per parti cruciali del testo (NEI) miste a supporti parziali, il sistema propende precauzionalmente per **Not Enough Information**.

Al termine dell’aggregazione, i risultati finali vengono passati al Supervisore che dichiara la fine del processo (FINISH), consentendo all’Orchestratore di salvare il documento finale in MongoDB e ag-

giornare l'interfaccia utente in tempo reale.

Chapter 6

Storage

6.1 Manuale d'Architettura: Storage e Data Provenance (MedFactCheck)

Il modulo di **Storage** è l'infrastruttura di persistenza dati del sistema MedFactCheck. Garantisce che l'intero ciclo di vita di un claim, dall'input iniziale fino al verdetto finale e alle evidenze recuperate, sia memorizzato in modo sicuro, interrogabile e trasparente.

6.2 Architettura del Database: MongoDB

Il sistema utilizza **MongoDB** come database NoSQL primario. La scelta di un database orientato ai documenti (schema-less) è perfetta per memorizzare strutture dati complesse, eterogenee e gerarchiche come quelle prodotte dal grafo multi-agente LangGraph e dai tool di

estrazione.

All'interno del database `medfactcheck`, vengono gestite tre collezioni principali (ottimizzate tramite la creazione automatica di indici all'avvio con `ensure_indexes()`):

6.2.1 **final_results** (Storico dei Claim e Verdetti)

Memorizza il risultato consolidato e lo stato di avanzamento di ogni singola verifica.

- **Struttura:** Contiene l'ID univoco, il testo originale, l'eventuale immagine allegata, il verdetto finale aggregato (es. *Supported*), il punteggio di confidenza medio, l'array dei *sub-verdicts* (ognuno con il relativo ragionamento logico CoT) e la traccia completa dell'esecuzione degli agenti (`agent_trace`).
- **Scopo:** Traccia lo stato della pipeline step-by-step (es. passando da *decomposed* a *retrieved*) e alimenta la Dashboard permettendo ricerche in tempo reale tramite Regex e filtri avanzati su data, fonti e verdetto.

6.2.2 **evidence** (Data Provenance e Tracciabilità)

Salva i singoli frammenti di testo (*chunk*) recuperati dalla letteratura o dalle Knowledge Base per ciascun sub-claim.

- **Struttura:** Ogni documento include il `claim_id` associato,

la fonte esatta (es. URL di Europe PMC, PubMed ID o DisGeNET), il testo del chunk e il punteggio di similarità calcolato in fase di retrieval.

- **Scopo:** Garantisce la rigorosa *Data Provenance*. L'utente finale (e la commissione) può sempre risalire all'abstract esatto o al paper che ha generato un determinato verdetto, requisito fondamentale in ambito clinico per disinnescare le allucinazioni dell'AI.

6.2.3 **papers** (Cache-Aside della Letteratura Europea)

Funge da layer di caching persistente e intelligente per i documenti XML integrali scaricati da Europe PMC.

- **Struttura:** Associa in modo univoco l'ID del paper al suo intero *Full-Text* estratto e pulito dai tag HTML/XML.
- **Scopo e Ottimizzazione:** Evita di scaricare ripetutamente lo stesso paper per claim diversi, azzerando la latenza di rete e proteggendo l'IP del server dal rate-limiting. A livello algoritmico, il Retriever interroga questa cache utilizzando l'operatore di Bulk-Read `$in`, recuperando decine di articoli in una singola, velocissima operazione di I/O.

6.3 Persistenza dello Stato del Grafo (Checkpointing)

Oltre ai dati di dominio, il sistema salva l'architettura di calcolo. Per gestire lo stato delle esecuzioni asincrone e i thread conversazionali, il sistema si avvale della libreria `langgraph-checkpoint-mongodb`. Questo meccanismo di “memory saver” cattura un'istantanea (*snapshot*) esatta dello stato del grafo (il `MultiAgentState`) al termine del ciclo di esecuzione di ogni singolo nodo (es. dopo che il `Decomposer` ha finito, prima che inizi il `Retriever`).

Questa funzionalità, unita alla generazione di un `thread_id` univoco all'avvio di ogni sessione, garantisce la **Fault-Tolerance**: in caso di spegnimento o riavvio improvviso del server, l'esecuzione della verifica può riprendere esattamente da dove si era interrotta senza subire alcuna perdita di dati.

6.4 Storage Vettoriale e Controllo d'Integrità (FAISS/BM25)

Parallelamente a MongoDB (che gestisce testi e metadati), il sistema mantiene indici vettoriali e lessicali locali tramite **FAISS** (Facebook AI Similarity Search) e **BM25Okapi** per l'interrogazione istantanea della Knowledge Base strutturata.

Per prevenire i comuni problemi di corruzione della cache vettoriale, lo storage fisico è stato blindato con due strategie:

- **Triplo Backup:** Gli indici (file `.index` e `.pkl`) vengono dumpati e serializzati in parallelo su tre percorsi distinti (es. `v1`, `v2`, `v3`).
- **Validazione Dimensionale (Fail-Fast):** Al riavvio del server, il sistema carica gli indici e verifica a basso livello che il numero di vettori (`idx.ntotal`) e di documenti (`bm25.corpus_size`) coincida perfettamente con la lunghezza dei record estratti da DisGeNET. Se vi è una discrepanza (dovuta magari ad un file corrotto), l'indice viene scartato, si tenta il ripristino dal backup e, in caso di totale fallimento, lo spazio vettoriale viene re-indicizzato dinamicamente in RAM per prevenire eccezioni *Out of Bounds* in fase di inferenza.

Chapter 7

Dashboard

7.1 MedFactCheck - Dashboard Client

Il modulo **Dashboard** costituisce l'interfaccia utente (*Front-End*) del sistema **MedFactCheck**. È un'applicazione web ultra-leggera e interattiva sviluppata in **Streamlit**, progettata per offrire un'esperienza utente reattiva e trasparente durante l'intero processo di fact-checking di claim biomedici.

7.2 Obiettivi e Responsabilità

In linea con l'architettura disaccoppiata del sistema, la Dashboard ha il compito esclusivo di gestire la presentazione dei dati e l'interazione con l'utente, delegando interamente il carico computazionale pesante (modelli IA, calcolo vettoriale, inferenza LLM) al server di *Back-End*

(Orchestratore).

Le sue responsabilità principali includono:

1. **Acquisizione dell'Input (Multimodale):** Raccogliere affermazioni sospette testuali (*claim*) e/o file multimediali (immagini biomediche, referti) inserite dall'utente, permettendo analisi puramente visive, puramente testuali o ibride.
2. **Delega al Back-End:** Inviare asincronamente il payload al server FastAPI (endpoint `/verify`) tramite chiamate HTTP POST (`requests.post`).
3. **Monitoraggio e Visualizzazione:** Interrogare localmente **MongoDB** per estrarre e mostrare i log e le decisioni prese dal Supervisore e dai nodi operativi (*Decomposer*, *Retriever*, *Reasoner*, *Veracity*).
4. **Rendering dei Risultati:** Visualizzare le metriche di output, la scomposizione dei *sub-claims*, i documenti recuperati (evidenze mediche) e la sentenza finale (es. *Supported*, *Refuted*, *Not Enough Info*) accompagnata dal punteggio di confidenza.
5. **Ricerca e Filtri Avanzati:** Esplorare l'intero database storico di MongoDB filtrando per parola chiave (Regex testuale), Verdetto, Fonte bibliografica (es. Europe PMC, DisGeNET) e Finestra Temporale (es. Ultime 24h, 7 giorni).

6. **Reportistica PDF:** Generare ed esportare dinamicamente un report PDF strutturato per ogni claim verificato, grazie all'integrazione con la libreria `reportlab`.

7.3 Struttura dell'Interfaccia (Layout e Navigazione)

L'interfaccia è suddivisa in aree logiche per massimizzare la *Usability* e la *Transparency*.

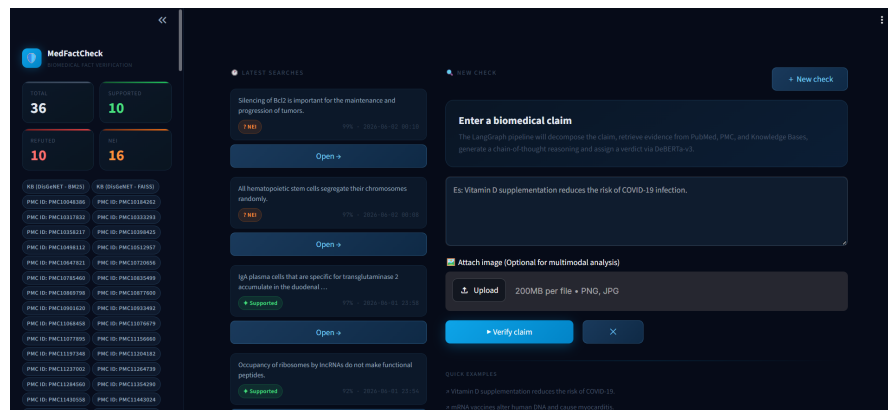


Figure 7.1: Panoramica dell'interfaccia utente (Dashboard) di MedFact Check, con Sidebar laterale per lo storico e pannello centrale di analisi.

- **Sidebar (Statistiche e Filtri):** Mostra un aggregato in tempo reale dei claim analizzati (*Total*, *Supported*, *Refuted*, *NEI*) per una visione globale. Permette di interrogare lo storico combinando filtri per Keyword, Verdetto, Fonte (es. Europe PMC, DisGeNET), Finestra Temporale e Soglia di Confidenza (Slider 0-100%).

- **Pannello Sinistro (Storico):** Una coda dinamica e scrollabile che mostra i claim più recenti e pertinenti ai filtri applicati, pronti per essere ricaricati e ispezionati istantaneamente.
- **Pannello Centrale (Input e Analisi):**
 - Presenta un form **Multimodale** per inserire sia testo libero/URL sia per caricare referti medici in formato immagine (PNG/JPG).
 - Durante l'esecuzione, un monitor dedicato mostra lo stato di avanzamento della pipeline LangGraph in esecuzione sul server.
 - A fine elaborazione, offre una visualizzazione a schede (*Radio Tabs*) per esplorare in profondità i risultati: **Verdict** (breakdown dei sub-claims), **Evidence** (paragrafi recuperati con link diretti ai paper originali), **Reasoning** (il log della *Chain-of-Thought*) e **Agent Trace** (i log dettagliati di *routing* del Supervisore e degli agenti).

7.4 Interazione Multimodale e Casi d'Uso

La flessibilità dell'Orchestratore e la natura multimodale del form di input della Dashboard permettono al sistema di adattarsi dinamicamente alla natura della richiesta. Di seguito si illustrano i quattro scenari principali di interazione supportati dall'applicativo:

7.4.1 Caso 1: Input Puramente Testuale (Testo Libero)

Quando l'utente inserisce esclusivamente un'affermazione biomedica nel campo di testo, la Dashboard impacchetta il payload come semplice stringa. Il nodo *Decomposer* estrae i sub-claim e avvia una classica pipeline RAG. L'interfaccia (Figura 7.2) si adatta mostrando direttamente le evidenze testuali recuperate dalla KB e il ragionamento deduttivo senza moduli visivi.

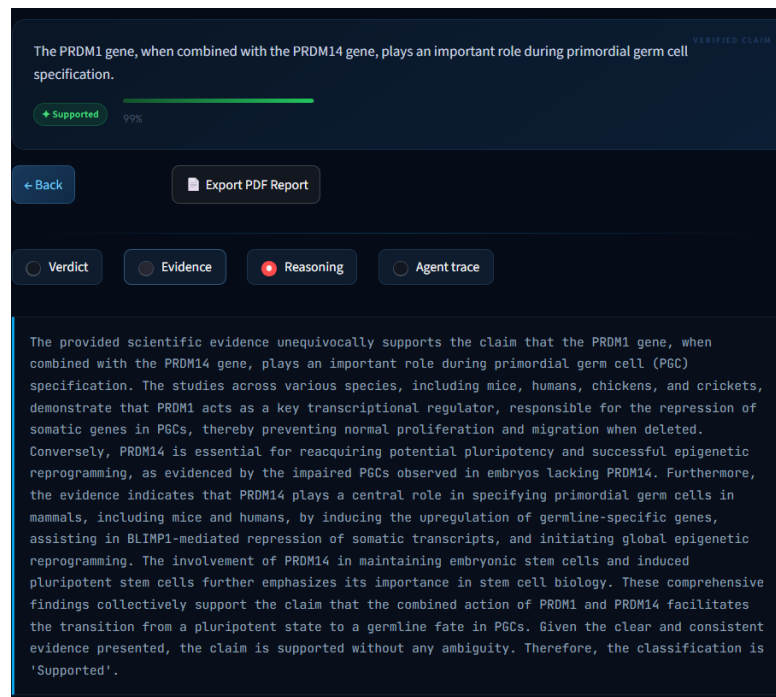


Figure 7.2: Reasoning.

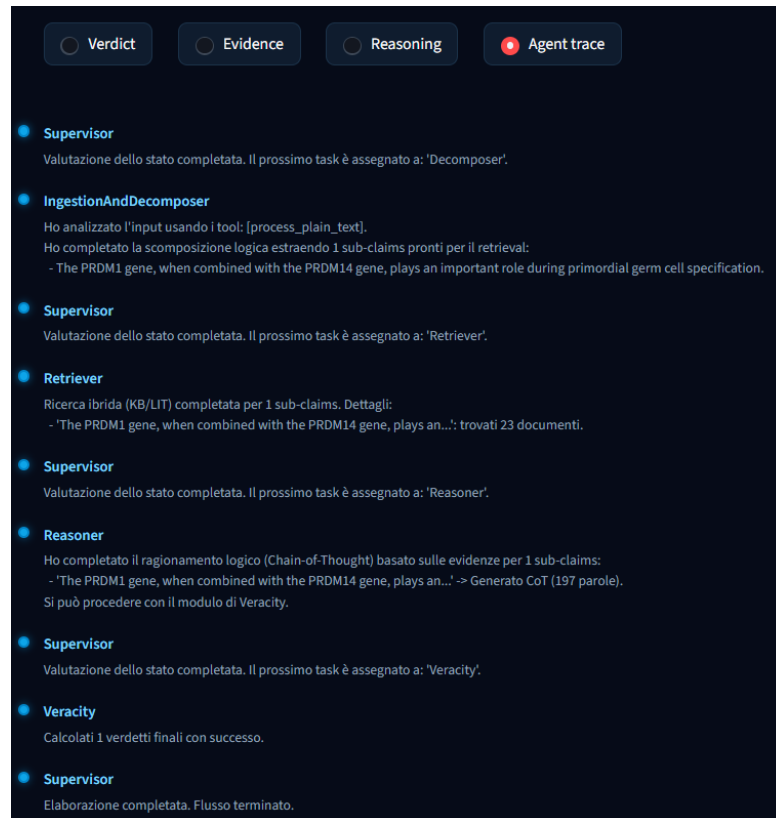


Figure 7.3: Agent Trace.

7.4.2 Caso 2: Input da Fonte Esterna (URL)

Qualora l'utente incolli il link a un articolo web o a un paper scientifico, la Dashboard lo trasmette all'API. Il back-end attiva il tool di web scraping autonomo (*scrape_text_from_url*), estrae il contenuto della pagina e lo decompone per la verifica. Questo permette all'utente di validare intere pagine web con un singolo click (Figura 7.4).

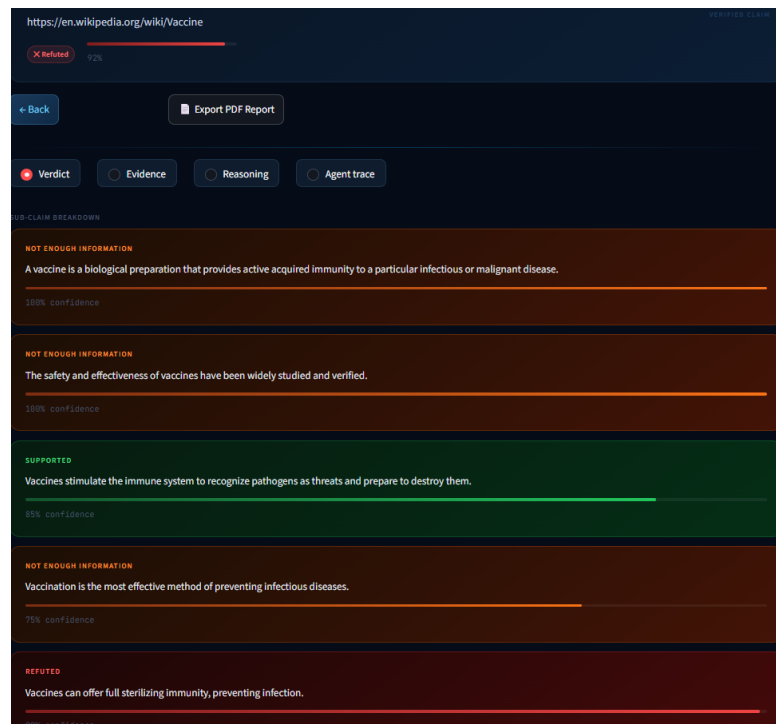


Figure 7.4: Verdict.

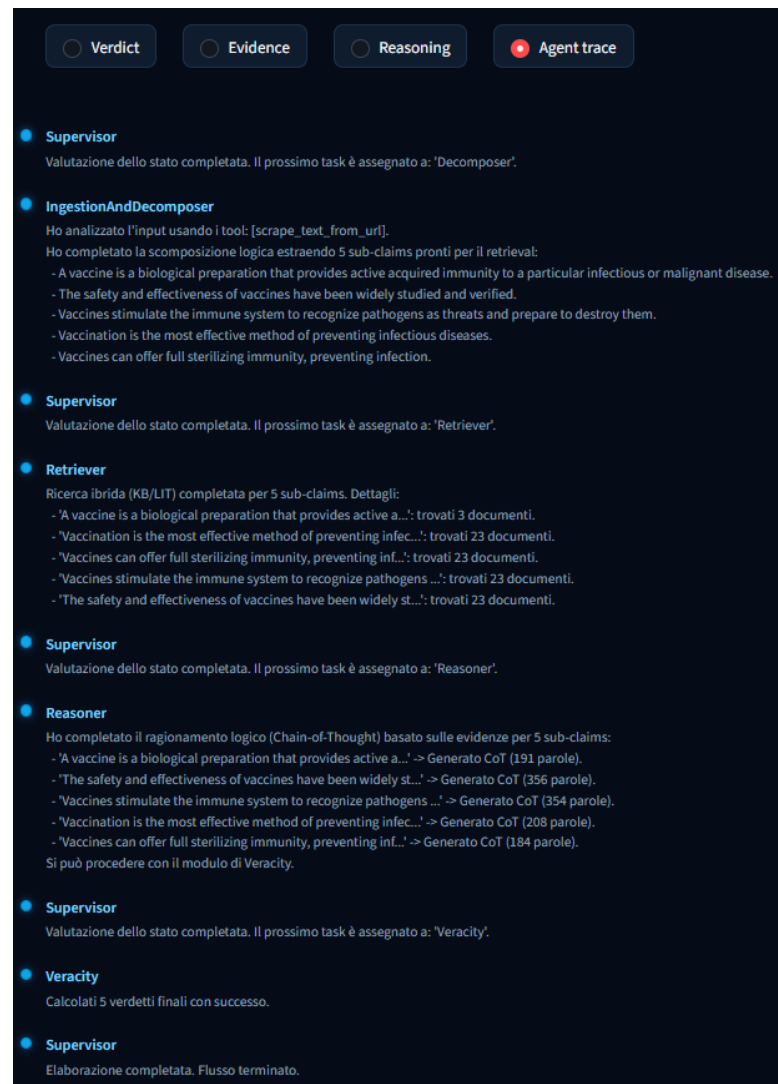


Figure 7.5: Agent trace.

7.4.3 Caso 3: Input Puramente Visivo

Caricando un referto medico o un'immagine diagnostica (es. un elettrocardiogramma o la foto di un neo) senza alcun testo di accompagnamento, l'interfaccia mostra un'anteprima dell'allegato. Lato server, il *Clinical Router* bypassa i tradizionali database testuali e sfrutta le capacità di Computer Vision del LLM per diagnosticare e refertare le

anomalie visive, producendo un report basato esclusivamente sull'ispezione dei pixel (Figura 7.6).

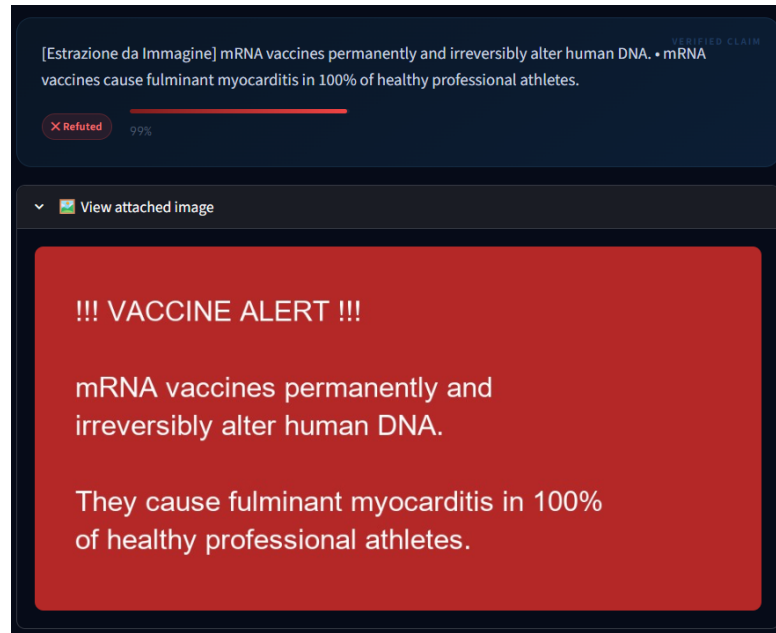


Figure 7.6: Verdict.

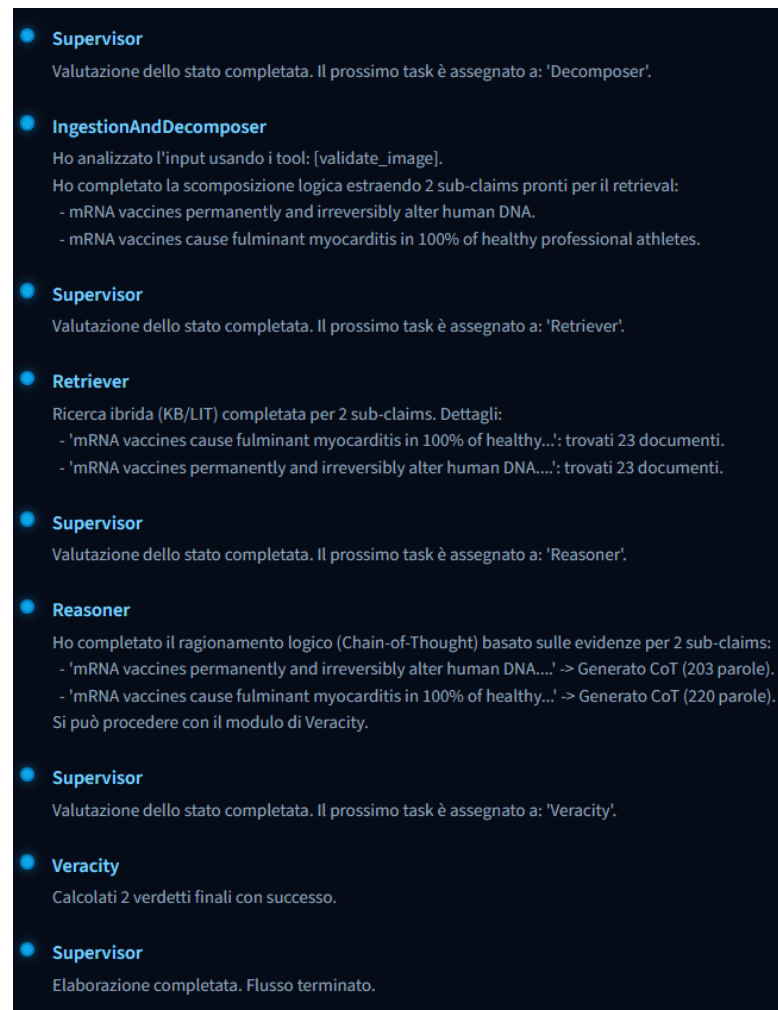


Figure 7.7: Agent trace.

7.4.4 Caso 4: Input Ibrido (Testo + Immagine)

Il caso d'uso più complesso si verifica quando l'utente fornisce un'immagine accompagnata da un claim testuale (es. una foto di un sospetto melanoma descritta nel testo come "neo innocuo"). La Dashboard trasmette entrambi i flussi di dati. Come mostrato in Figura 7.8, il modello multimodale analizza la discrepanza, annulla il *Few-Shot Bias* testuale, smentisce l'affermazione fuorviante dell'utente e genera

un'analisi basata sulla reale patologia osservata nell'immagine.

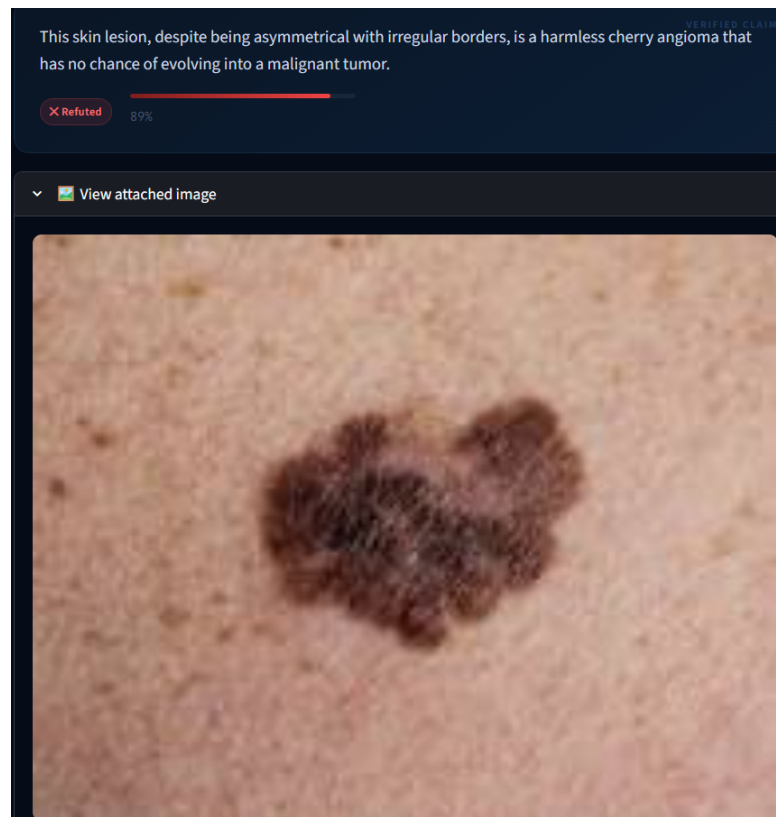


Figure 7.8: Input.

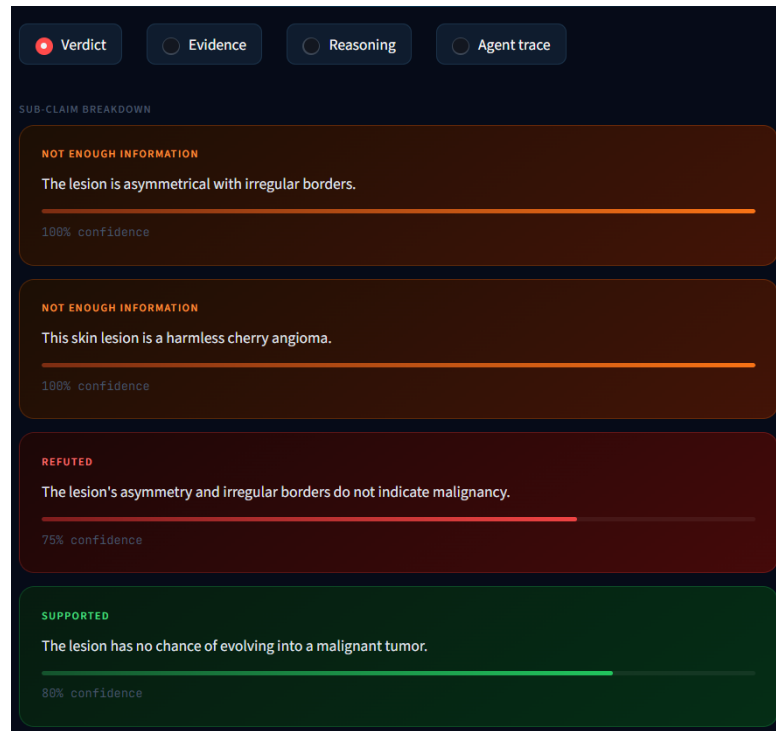


Figure 7.9: Verdict.

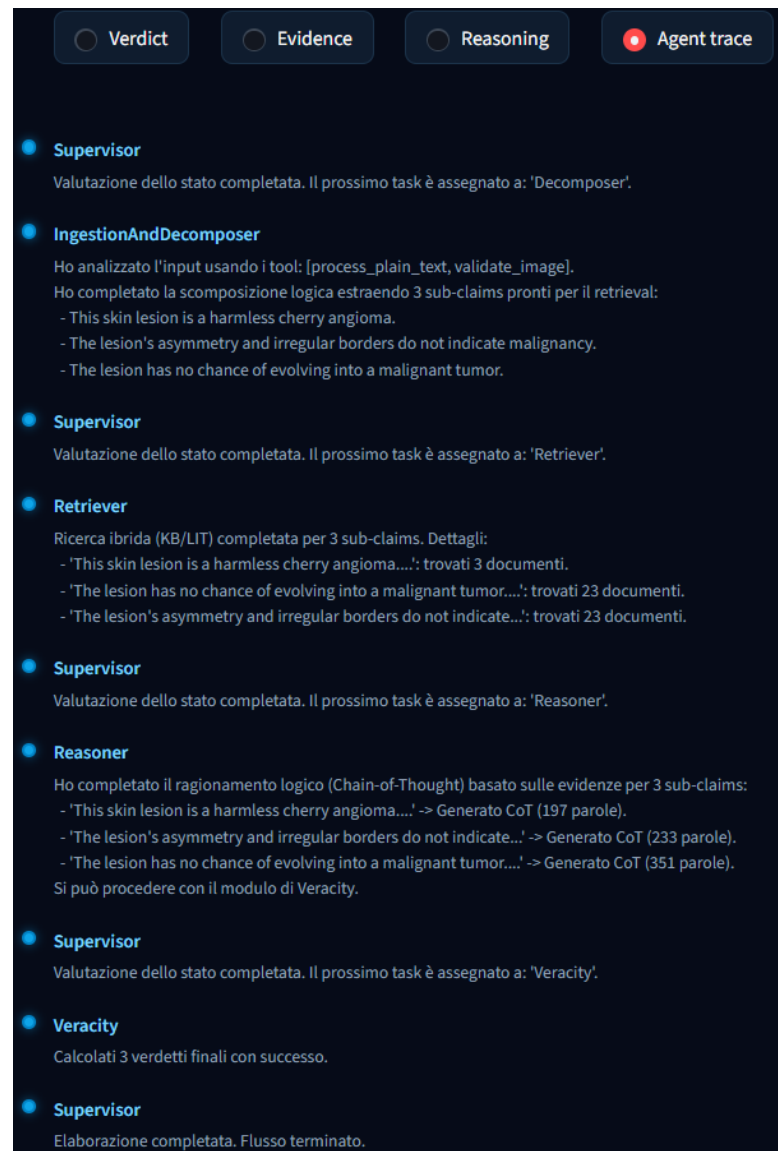


Figure 7.10: Agent trace.

7.5 Gestione degli Errori (Defensive Programming)

Il *Front-End* è ingegnerizzato per resistere a eventuali instabilità del *Back-End*. Include controlli di *Defensive Programming* per gestire:

- **LLM Token Limit / Truncation:** Se il modello generativo (es. Qwen) si interrompe bruscamente per aver raggiunto il limite di token senza restituire un JSON valido, la Dashboard rileva elegantemente l'assenza del verdetto evitando crash applicativi (come i classici `KeyError`) e avvisando l'utente.
- **Timeout e I/O Sicuro:** Gestione sicura delle disconnessioni e rimozione deterministica in blocco `finally` delle immagini temporanee salvate in RAM per evitare *memory leaks* lato client.

7.6 I Vantaggi del Disaccoppiamento

Mantenere la Dashboard Streamlit come client separato assicura enormi vantaggi in termini di **Scalabilità**:

- L'interfaccia occupa risorse di calcolo minime e garantisce un'alta responsività della UI.
- La VRAM necessaria per i modelli (Qwen2.5, DeBERTa, indici FAISS) non è intaccata né frammentata da operazioni di UI.
- Decine di utenti possono avviare e usare la Dashboard contemporaneamente, sfruttando le code asincrone dell'Orchestratore, senza il rischio di bloccare il rendering a schermo.

7.7 Strategia di Caching (Smart Keying)

Per prevenire il sovraccarico del database e garantire un'esperienza utente istantanea, la Dashboard implementa una rigorosa politica di **Caching Parametrico**:

- Le query effettuate a MongoDB vengono memorizzate in RAM (tramite `@st.cache_data`) con un **Time-to-Live (TTL)** di **1 ora**.
- I risultati sono legati alla specifica combinazione di filtri richiesti dall'utente. Questo significa che navigare tra le varie schermate di un claim (Verdetto, Evidenze, Log) non scatuisce nessuna nuova query al DB, servendo i dati in pochi millisecondi.
- **Invalidazione Automatica**: Al momento della sottomissione di un *nuovo* claim verso l'API, la cache viene resettata dinamicamente (`st.cache_data.clear()`), garantendo che l'interfaccia mostri immediatamente il nuovo dato processato a tutti gli utenti attivi.

Chapter 8

Valutazione del Sistema e Conclusioni

8.1 Metodologia di Benchmarking

In conformità ai requisiti di valutazione quantitativa del modulo di *Veracity Prediction*, le performance architetturali e decisionali di Med-Fact Check sono state misurate utilizzando il dataset benchmark **Sci-Fact** (sviluppato dall'istituto di ricerca AllenAI).

A causa delle restrizioni di sicurezza nell'esecuzione di script remoti su piattaforme cloud (es. limitazioni di rete in Colab), l'infrastruttura di test è stata ingegnerizzata per scaricare, estrarre e parsare offline il dataset. Il benchmark è stato condotto in modalità end-to-end, misurando metriche di classificazione multiclasse (Precision, Recall e F1-Score) per le tre categorie previste dal sistema: *Supported*, *Refuted*

e *Not Enough Information (NEI)*.

8.2 L’Effetto Pendolo nel Prompt Engineering

Per valutare oggettivamente l’impatto delle istruzioni di sistema sulle performance decisionali dell’Agente *Reasoner*, è stato condotto un Testing sui *Prompt*. Al fine di isolare la variabile linguistica, è stato impostato un seme crittografico fisso (`random.seed(42)`) nello script di test. Questo ha garantito che, ad ogni iterazione, il modello affrontasse lo stesso identico sottoinsieme di claim estratti da SciFact, azzerando le fluttuazioni statistiche dovute alla difficoltà intrinseca dei singoli testi.

I log di esecuzione (riassunti nella Tabella 8.1 a fine capitolo) hanno evidenziato il cosiddetto “Effetto Pendolo”, dimostrando in modo empirico la vulnerabilità semantica dei Large Language Models. Di seguito si riportano per esteso i payload testuali (*System Message* e *User Message*) utilizzati per le quattro iterazioni di test.

1. Prompt Severo

Nella prima iterazione, l’istruzione chiedeva un’analisi logica imparziale. In assenza di linee guida sulla flessibilità semantica, il modello ha adottato un approccio pedante (*Strict Lexical Matching*). Non trovando

CHAPTER 8. VALUTAZIONE DEL SISTEMA E CONCLUSIONI

corrispondenze lessicali esatte nei paper (fortemente parafrasati rispetto ai claim), l'agente ha ritenuto insufficiente ogni prova fornita, classificando l'intero dataset come *Not Enough Information* e azzerando le metriche sulle classi attive.

System Role:

You are a rigorous, verbose, and impartial scientific analyst. Your goal is to explain the logical reasoning in detail.

User Prompt:

You are an expert biomedical analyst tasked with validating scientific claims.

Evaluate the following claim by rigorously comparing it with the provided evidence.

CLAIM TO VERIFY: "{sub_claim}"

SCIENTIFIC EVIDENCE FOUND:

{evidence_text}

REASONING INSTRUCTIONS (Chain-of-Thought):

Write a detailed and discursive analysis. In your text you must:

1. Synthesize what the provided scientific evidence clearly states.
2. Explicitly relate the concepts of the evidence to the claim.

3. Conclude unequivocally whether the evidence supports, refutes, or does not offer sufficient details to confirm the claim.

Do not use bullet points, write a logical and cohesive paragraph.

DETAILED LOGICAL ANALYSIS:

2. Prompt Permissivo

Per ovviare alla rigidità del test iniziale, è stata iniettata una direttiva di tolleranza semantica tramite la clausola *CRITICAL INSTRUCTION*. Come evidenziato dai dati, il modello si è parzialmente sbloccato iniziando a riconoscere alcune inferenze positive (la Recall per *Supported* è passata dallo 0% al 33%). Tuttavia, l'eccessiva flessibilità ha compromesso totalmente la capacità di rilevare le palesi contraddizioni cliniche, azzerando le performance sulla classe *Refuted*.

System Role:

You are a rigorous but flexible scientific analyst. Your goal is to explain the logical reasoning in detail, focusing on clinical meaning rather than exact lexical matches.

User Prompt:

You are an expert biomedical analyst tasked with validating scientific claims.

CHAPTER 8. VALUTAZIONE DEL SISTEMA E CONCLUSIONI

Evaluate the following claim by rigorously comparing it with the provided evidence.

CLAIM TO VERIFY: "{sub_claim}"

SCIENTIFIC EVIDENCE FOUND:

{evidence_text}

REASONING INSTRUCTIONS (Chain-of-Thought):

Write a detailed and discursive analysis. In your text you must:

1. Synthesize what the provided scientific evidence clearly states.
2. Explicitly relate the concepts of the evidence to the claim.
3. Conclude unequivocally whether the evidence supports, refutes, or does not offer sufficient details to confirm the claim.

CRITICAL INSTRUCTION: Be a flexible clinical analyst. If the evidence supports the general clinical meaning of the claim, even using synonyms or omitting minor details, clearly state that the evidence **SUPPORTS** the claim. Do not require exact lexical matching.

Do not use bullet points, write a logical and cohesive paragraph.

DETAILED LOGICAL ANALYSIS:

3. Prompt Cautelativo Estremo

Per arginare le derive deduttive del test precedente, è stato imposto un vincolo logico restrittivo (operatore *MUST*). Questo comando categorico ha mandato in “tilt cautelativo” il modello: per non violare il divieto di deduzione, l’agente si è rifiutato di emettere verdetti clinici, classificando sistematicamente tutto come *NEI* (Recall 100%) e replicando il collasso decisionale del primo test.

System Role:

```
You are a rigorous, verbose, and impartial scientific
analyst. Your goal is to explain the logical reasoning
in extreme detail. You focus on clinical meaning but
strictly require direct evidence to draw conclusions.
```

User Prompt:

```
You are an expert biomedical analyst tasked with
validating scientific claims.
Evaluate the following claim by rigorously comparing it
with the provided evidence.

CLAIM TO VERIFY: "{sub_claim}"

SCIENTIFIC EVIDENCE FOUND:
{evidence_text}

REASONING INSTRUCTIONS (Chain-of-Thought):
Write a highly detailed and verbose discursive analysis (
```

about 150–200 words). In your text you must:

1. Synthesize what the provided scientific evidence clearly states.
2. Explicitly relate the concepts of the evidence to the claim.
3. Conclude unequivocally whether the evidence supports, refutes, or does not offer sufficient details to confirm the claim.

CRITICAL INSTRUCTION: You are a rigorous medical analyst.

If the retrieved evidence clearly confirms or refutes the clinical intent of the claim (even using synonyms), explicitly state that it supports or refutes it.

HOWEVER, if the evidence discusses a related topic but DOES NOT provide explicit and direct proof to confirm the user's specific statement, you MUST conclude that there is 'Not Enough Information'. Do not guess or make deductions without direct evidence.

Do not use bullet points, write a logical, verbose, and cohesive paragraph.

DETAILED LOGICAL ANALYSIS:

4. Calibrazione Bilanciata

Nell'iterazione finale, i vincoli assoluti sono stati sostituiti da un set di definizioni condizionali chiare (operatore *ONLY IF*), mitigando il ruolo della classe NEI come via di fuga. Questa architettura ha sbloccato il

CHAPTER 8. VALUTAZIONE DEL SISTEMA E CONCLUSIONI

modello, ristabilendo predizioni strutturate (es. recupero dell’F1 sui *Refuted* fino a 0.57).

System Role:

You are a rigorous, verbose, and impartial scientific analyst. Your goal is to explain the logical reasoning in extreme detail, focusing on the clinical core of the claim.

User Prompt:

You are an expert biomedical analyst tasked with validating scientific claims.
Evaluate the following claim by rigorously comparing it with the provided evidence.

CLAIM TO VERIFY: "{sub_claim}"

SCIENTIFIC EVIDENCE FOUND:

{evidence_text}

REASONING INSTRUCTIONS (Chain-of-Thought):

Write a highly detailed and verbose discursive analysis (about 150–200 words). In your text you must:

1. Synthesize what the provided scientific evidence clearly states.
2. Explicitly relate the concepts of the evidence to the claim.
3. Conclude unequivocally whether the evidence supports, refutes, or does not offer sufficient details to

confirm the claim.

CRITICAL INSTRUCTION: You are a medical classifier. Your task is to evaluate whether the provided evidence confirms or refutes the claim.

- Use 'Supported' if the evidence clearly confirms the clinical core of the claim (even if using synonyms).
- Use 'Refuted' if the evidence clearly contradicts the claim.
- Use 'Not Enough Information' ONLY IF the evidence is vague, irrelevant, or lacks the specific details to confirm the user's statement.

Do not use bullet points, write a logical, verbose, and cohesive paragraph.

DETAILED LOGICAL ANALYSIS:

L'Accuracy globale stabilizzata al 40% nel test definitivo certifica, infine, un limite architetturale importante: in un approccio *zero-shot*, la sola ottimizzazione ingegneristica del prompt non è sufficiente a compensare la profondità logica richiesta dai claim altamente parafrasati della letteratura medica, evidenziando la necessità futura di un *fine-tuning* dei pesi del classificatore NLI.

Table 8.1: Testing sui Prompt (Campione fisso da 10 claim): Precision (P), Recall (R) e F1-Score (F1)

Variante Prompt	Acc.	Supported			Refuted			NEI			Esito Comportamentale
		P	R	F1	P	R	F1	P	R	F1	
1. Severo	30%	0.00	0.00	0.00	0.00	0.00	0.00	0.33	1.00	0.50	Severità estrema: non trova match lessicali e classifica tutto come NEI.
2. Permissivo	30%	0.25	0.33	0.29	0.00	0.00	0.00	0.40	0.67	0.50	Sblocco sui Supported, ma totale incapacità di rilevare i claim Refuted.
3. Cautelativo	30%	0.00	0.00	0.00	0.00	0.00	0.00	0.33	1.00	0.50	La classe NEI assorbe sistematicamente tutto.
4. Bilanciato	40%	0.00	0.00	0.00	0.67	0.50	0.57	0.33	0.67	0.44	Recupero sui Refuted, ma limite zero-shot fisiologico sui claim positivi.

**Nota: Il sottoinsieme isolato dal seed contiene 3 claim Supported, 4 Refuted e 3 NEI. Pertanto, i valori metrici nulli (0.00) non derivano dall'assenza statistica del target nel campione, ma dall'incapacità intrinseca del modello zero-shot di cogliere l'inferenza clinica.*

8.3 Conclusioni

Il progetto MedFact Check risponde con successo all'esigenza di un sistema automatizzato e interpretabile per la validazione della letteratura biomedica. L'implementazione di un'architettura Multi-Agente (Hub & Spoke) disaccoppiata ha permesso di orchestrare in modo fluido l'ingestione multimodale, la ricerca RAG e la verifica inferenziale.

L'uso sinergico di LLM (Qwen2.5-VL in formato NF4) per il reasoning e modelli NLI densi (DeBERTa) per il verdetto quantitativo ha consentito di mitigare in modo severo il rischio di allucinazioni mediche. Inoltre, l'adozione del database MongoDB come *Cache-Aside*

e l'impiego di indici vettoriali FAISS garantiscono il funzionamento in ambienti con risorse limitate preservando rigorosamente la *Data Provenance*.